Day 10 – Testing & Deployment

🏷️ Setup & Prerequisites

1. **Setup Checklist**:
   - **IDE Ready**: Open VS Code, prepared to inspect `.spec.ts` testing files and build scripts.
   - **Terminal Active**: Ensure you can run Angular CLI build and test commands in the terminal window.
2. **Prerequisites**:
   - You must have completed Day 9 successfully (understanding component parameters, inputs, outputs, and template pipelines).

🎯 Learning Objectives

By the end of this class, you will be able to:

- Explain testing principles and configure Angular test suites using `TestBed`.
- Write unit tests for services using `HttpClientTestingModule` or mock HTTP providers.
- Write component unit tests to verify HTML template renders and form inputs validation.
- Execute test suites and collect coverage reports using the Angular CLI.
- Build optimized production-ready static bundles using `ng build`.
- Deploy built static assets to hosting platforms like Vercel, Netlify, or Firebase Hosting.

⏱️ Classroom Timeline (45 min)

Segment	Duration	Focus
1. Hook & Big Picture	9 min	Introduce frontend testing. Discuss local dev server execution vs. static CDN production setups.
2. Key Concepts & Core Ideas	7 min	Explain TestBed configuration, spies, build optimization flags, and CDN static hosting.
3. Live Coding Walkthrough	20 min	Write service tests, write component form validation tests, compile production files, and configure deploy redirects.
4. Check for Understanding	4 min	Q&A on mocking backend requests, differential loading, and production SPA route redirects.

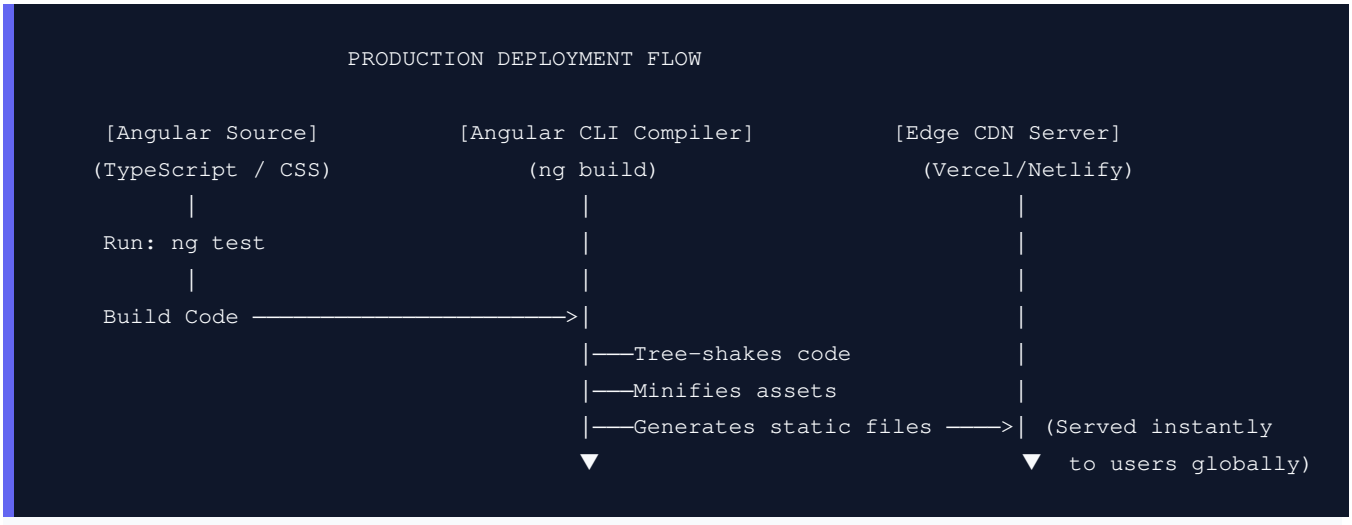
5. Class Wrap-up & Wrap-up Checklist	5 min	Complete final system integration checklist. Celebrate Angular course completion!

🚀 Step-by-Step Walkthrough

1. Hook & Big Picture (9 min)

How do we guarantee that adding a new styling module or routing path today doesn't break our Day 5 reactive form validations? If we have to click through the user registration and product forms manually on every edit, we waste hours of development time. We write automated unit tests to verify our code logic in milliseconds.

- **Production Build Lifecycle**: We also need to package our application for release. Unlike Python backends which execute source scripts dynamically on a server, Angular is a client-side library. When we run a production build, the Angular compiler tree-shakes (deletes unused code), minifies, and compiles all assets into a few highly optimized static HTML, CSS, and Javascript files that are served to users from rapid Content Delivery Networks (CDNs).



2. Key Concepts & Core Ideas (7 min)

Introduce these testing and compilation terms:

- **Jasmine & Karma**: The default test tools in Angular. Jasmine is the test framework where you write test suites (`describe`, `it`, `expect`). Karma is the test runner that opens a browser window and runs the test suites, showing live results.
- **`TestBed`**: The core Angular utility for testing. It behaves like a virtual Angular module container, letting you configure imports and register dependencies to compile components and services in isolation.
- **`spyOn()`**: A Jasmine feature that creates a mock "spy" on a method. It lets you monitor if a method was called, intercept parameters, and return fake mock results.

- **Tree Shaking**: A compilation step that analyzes import statements and strips out unused functions from the final compiled bundle, reducing page load sizes.
- **AOT (Ahead-of-Time) Compilation**: The compiler translates HTML templates and TypeScript code into browser-executable Javascript *during* the build phase, rather than having the browser compile templates on-the-fly, leading to faster startup times.

---

3. Live Coding Walkthrough (20 min)

Step 3.1: Writing Unit Tests for GameService

- **Concept**: Open `src/app/services/game.service.spec.ts`. We will write a unit test suite to verify that `getGames()` makes an HTTP GET request to the backend and returns the games list successfully.
- **Code**: Modify `src/app/services/game.service.spec.ts` (using HttpTestingController to mock API responses):

```
import { TestBed } from '@angular/core/testing';
import { provideHttpClient } from '@angular/common/http';
import { HttpTestingController, provideHttpClientTesting } from '@angular/common/http/testin
import { GameService, Game } from './game.service';

describe('GameService', () => {
  let service: GameService;
  let httpMock: HttpTestingController;

  // TestBed configures the testing environment before each test runs
  beforeEach(() => {
    TestBed.configureTestingModule({
      providers: [
        GameService,
        provideHttpClient(),
        provideHttpClientTesting() // Mock provider to intercept HTTP calls
      ]
    });
    service = TestBed.inject(GameService);
    httpMock = TestBed.inject(HttpTestingController);
  });

  // Verify all mock HTTP expectations are satisfied after each test
  afterEach(() => {
    httpMock.verify();
  });

  it('should be created', () => {
    expect(service).toBeTruthy();
  });

  it('should fetch games list via HTTP GET', () => {
    const mockGames: Game[] = [
      { id: 1, title: 'Portal 3', price: 29.99 }
```

```
    ];

    // Call the service and check the outputs when the stream resolves
    service.getGames().subscribe(games => {
      expect(games.length).toBe(1);
      expect(games[0].title).toBe('Portal 3');
    });

    // Expect a GET request to the target API URL
    const req = httpMock.expectOne('http://127.0.0.1:8000/api/games/');
    expect(req.request.method).toBe('GET');

    // Flush mock data to resolve the HTTP request stream
    req.flush(mockGames);
  });
});
```

- **Verify**: Run `ng test` in the terminal to verify the tests compile and pass.

Step 3.2: Writing Unit Tests for AddGameFormComponent

- **Concept**: Open `src/app/components/add-game-form/add-game-form.spec.ts`. We will write component tests to verify that validation rules block form submission if fields are empty.
- **Code**: Modify `src/app/components/add-game-form/add-game-form.component.spec.ts`:

```
import { ComponentFixture, TestBed } from '@angular/core/testing';
import { AddGameFormComponent } from './add-game-form.component';
import { ReactiveFormsModule } from '@angular/forms';
import { provideHttpClient } from '@angular/common/http';
import { provideHttpClientTesting } from '@angular/common/http/testing';
import { provideRouter } from '@angular/router';

describe('AddGameFormComponent', () => {
  let component: AddGameFormComponent;
  let fixture: ComponentFixture<AddGameFormComponent>;

  beforeEach(async () => {
    await TestBed.configureTestingModule({
      imports: [AddGameFormComponent, ReactiveFormsModule],
      providers: [
        provideHttpClient(),
        provideHttpClientTesting(),
        provideRouter([]) // Mock router dependency
      ]
    }).compileComponents();

    // Component fixture handles component instances and DOM inspection
    fixture = TestBed.createComponent(AddGameFormComponent);
    component = fixture.componentInstance;
    fixture.detectChanges(); // Trigger Angular change detection cycle
  });

  it('should initialize form controls', () => {
```

```
    expect(component.gameForm).toBeDefined();
    expect(component.title?.value).toBe('');
    expect(component.price?.value).toBe(0);
  });

  it('should show form invalid status on empty fields', () => {
    // Inputs are empty by default, form should be invalid
    expect(component.gameForm.valid).toBeFalse();
  });

  it('should validate title custom uppercase rule', () => {
    const titleControl = component.title;

    // Set lowercase title value
    titleControl?.setValue('portal 3');
    expect(titleControl?.valid).toBeFalse();
    expect(titleControl?.errors?.['titleLowercase']).toBeTrue();

    // Set valid uppercase value
    titleControl?.setValue('Portal 3');
    expect(titleControl?.valid).toBeTrue();
  });
});
```

- **Verify**: Verify that all tests pass cleanly in the running Karma browser dashboard.

Step 3.3: Production Compilation Build

- **Concept**: Compile and build your application for release.
- **Code**:

```
# Compile production bundles
ng build
```

- **Verify**: Confirm that a `dist/gamekey-store/browser/` folder is generated in your project root containing compiled `index.html`, CSS assets, and minified JS bundles.

Step 3.4: Deploying to Hosting Platforms

- **Concept**: Static CDNs (like Netlify or Vercel) serve the compiled files. We must configure redirection rules so client-side navigation route refreshes (like `/games/add`) don't crash with 404s.
- **Redirection Setup**:
  - **Netlify**: Create a `_redirects` file in `src/public/` (or specify in netlify.toml):

    ```
    /*    /index.html   200
    ```

    This redirects all unknown URL routes to the shell index.html, allowing the Angular Router to handle them.

- **Vercel**: Create a configuration file in the project root folder:

```
{
  "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }]
}
```

- **Deploy Command**: Push your repository to GitHub, link the repository on Vercel or Netlify, specify build output directory as `dist/gamekey-store/browser`, and deploy!

---

4. Check for Understanding (4 min)

> ***Question**: "Why do we call `fixture.detectChanges()` inside component tests?"*
> ***Answer**: In test suites, Angular's automatic Zone.js change detection is disabled. To force the component to compile its templates, bind its variable values, and update DOM nodes, you must call `fixture.detectChanges()` manually.*

> ***Question**: "What is the purpose of the `HttpTestingController` inside service tests?"*
> ***Answer**: It is a mock client that intercepts HTTP requests sent by Angular's `HttpClient`. It prevents real network requests from hitting backend API databases, allowing tests to check the outgoing HTTP URL/method parameters and flush fake mock responses instantly.*

> ***Question**: "What is the role of a redirection file (like `vercel.json` or `_redirects`) when deploying SPAs?"*
> ***Answer**: SPAs use client-side routing. If a user navigates to `/games/add` and refreshes the browser, the hosting server checks its directory for a file named `games/add`, finds nothing, and returns a 404 error. The redirection rules tell the server to route all requested paths to the root `/index.html` file, permitting the Angular Router inside the browser to parse the path and load the correct view.*

---

5. Final Integration Checklist (5 min)

Final checklist before wrapping up:

- [ ] **Components Layout**: Catalog lists and dynamic detail cards load and render without console errors.
- [ ] **Form Validations**: Built-in and custom uppercase validators block invalid form submits.
- [ ] **State Engine**: CENTRALIZED cart service syncs quantities and costs across components via Signals.
- [ ] **HTTP Requests**: HttpClientGET/POST calls communicate with backend databases successfully.

- [ ] **Unit Tests**: The test suite runs successfully and verifies both code calculations and validation rules.
- [ ] **Production Build**: `ng build` generates optimized static bundles successfully.
- [ ] **SPA Redirects**: Rewrite configuration rules protect routes from browser-refresh 404 crashes.